

BOOK 7 — JDBC, ORM, HIBERNATE & MAVEN

Complete Fresher / Entry-Level Java Full Stack Preparation Series

CHAPTER 1 — WEB ARCHITECTURE & JDBC FUNDAMENTALS

1.1 Java Editions Overview (JSE / JEE / JME / JavaFX)

Java is categorized into distinct platforms targeting specific application domains:

- **Java SE (Java Standard Edition - JSE):** Core Java platform containing foundational APIs (`java.lang`, `java.util`, `java.io`, `java.net`). Used for desktop applications and standalone logic.
- **Java EE (Java Enterprise Edition - JEE / Jakarta EE):** Built on top of Java SE for enterprise, web, and distributed applications. Includes Servlet, JSP, EJB, JPA, and RESTful web services.
- **Java ME (Java Micro Edition - JME):** Resource-constrained platform designed for embedded systems, set-top boxes, and legacy mobile devices.
- **JavaFX:** Client application platform for creating modern, rich desktop and cross-platform Graphical User Interfaces (GUIs).

1.2 Application Types (Desktop, Web, Client/Server)

- **Desktop Applications:** Standalone GUI or CLI applications executing on a single local machine (e.g., media players, IDEs).
- **Web Applications:** Applications running on remote servers accessed by client web browsers over HTTP/HTTPS protocols (e.g., e-commerce portals, banking applications).
- **Client/Server Applications:** Distributed systems where client applications send requests across a network to dedicated server applications that process requests and manage resources.

1.3 Web Application Architecture

Modern enterprise applications follow a **Multi-Tier (Layered) Architecture**:

1. **Presentation Layer (Frontend):** Client UI created using HTML, CSS, JavaScript, React, Angular, or JSP.
2. **Business Logic Layer (Backend):** Server-side Java code handling business rules, validation, and processing (Servlets, Spring Boot, POJOs).
3. **Data Access Layer (Persistence):** Server-side components managing interaction with databases (JDBC, Hibernate, Spring Data JPA).
4. **Database Layer (Database):** Relational (RDBMS) or NoSQL databases storing persistent application data (MySQL, PostgreSQL, Oracle).

1.4 Layered Components: Frontend / Backend / Database

- **Frontend:** Captures user input and renders visual responses.

- **Backend:** Receives client requests, processes business logic, and delegates data persistence operations.
- **Database:** Stores application state durably across application restarts in tables containing rows and columns.

1.5 Request–Response Cycle

1. **Request:** User performs an action on the client interface; the browser constructs an HTTP request containing parameters/payload and sends it to the backend server.
2. **Processing:** Backend controllers receive the request, invoke service layer methods, execute database queries via JDBC/ORM, and receive database result sets.
3. **Response:** Backend converts domain data into an HTTP response (HTML, JSON, XML) and transmits it back to the client browser for rendering.

1.6 JDBC Definition & Full Form

JDBC stands for **Java Database Connectivity**. It is a standard Java API ([java.sql](#) and [javax.sql](#) packages) provided by Oracle/Sun to interact with Relational Database Management Systems (RDBMS) from Java applications.

1.7 Need for JDBC

- **Database Neutrality:** Without JDBC, Java applications would require vendor-specific APIs to talk to different databases (MySQL, Oracle, PostgreSQL).
- **Standardized API:** JDBC provides a unified set of interfaces; developers write Java code against [java.sql](#) interfaces regardless of the underlying database vendor.
- **Persistence:** Enables Java programs to save, retrieve, update, and delete application data permanently in relational tables.

1.8 JDBC Architecture

JDBC architecture follows a two-tier and three-tier processing model:

- **Application Layer:** Java application calling standard JDBC API interfaces ([Connection](#), [Statement](#), [ResultSet](#)).
- **Driver Manager Layer:** [java.sql.DriverManager](#) tracking and managing vendor database drivers.
- **Driver Layer:** Vendor-specific driver implementation translating standard JDBC calls into database-native protocol commands.
- **Database Layer:** Relational database server executing SQL commands and returning tabular data.

1.9 JDBC Detailed Architecture & Component Roles (Syllabus 25.48)

1. **JDBC API:** Standard interfaces and classes in [java.sql](#) and [javax.sql](#).
2. **JDBC Driver Manager:** Discovers, registers, and manages database drivers; negotiates connections using database URLs.
3. **JDBC Test Suite:** Validates that a database driver complies with JDBC specifications.
4. **JDBC-ODBC Bridge (Legacy):** Historical bridge translating JDBC calls to ODBC calls (removed in Java 8).

1.10 JDBC Driver Types (1 to 4) (Syllabus 25.49)

JDBC drivers are classified into 4 distinct types based on implementation architecture:

1. **Type 1 Driver (JDBC-ODBC Bridge):** Converts JDBC calls to ODBC calls. Requires ODBC driver installation on client machine. Obsolete and removed in Java 8.
 2. **Type 2 Driver (Native-API Driver / Partly Java):** Converts JDBC calls to native C/C++ database client library calls. Requires native client library installation on target machine.
 3. **Type 3 Driver (Network-Protocol Driver / Middleware):** Converts JDBC calls to a middleware server protocol, which converts calls to database protocols. Flexible for multi-database environments.
 4. **Type 4 Driver (Native-Protocol Driver / Pure Java):** Converts JDBC calls directly into vendor-specific database network protocols without requiring client-side native library installation. Widely used because it communicates directly with the database using its native protocol, offering high performance and platform independence.
-

1.11 Core JDBC API Interfaces Overview

The `java.sql` package contains key interfaces:

- **Connection:** Represents an active session/connection with a specific database.
- **Statement:** Executes static SQL queries without parameter substitution.
- **PreparedStatement:** Used to execute parameterized SQL statements using `?` placeholders. Parameter values are supplied separately from the SQL text, which helps prevent SQL injection when used correctly. Preparation and execution-plan reuse depend on the JDBC driver and database.
- **CallableStatement:** Executes database stored procedures and functions.
- **ResultSet:** A tabular data stream representing query results returned by `SELECT` operations.
- **ResultSetMetaData:** Provides structural information about a `ResultSet` (column names, column types, column counts).
- **DatabaseMetaData:** Provides metadata about the database server itself (product name, version, supported SQL syntax).
- **SQLException:** Exception class thrown when database access errors or SQL syntax errors occur.

1.12 JDBC Programming Steps Overview

Standard JDBC execution involves 5 fundamental steps:

1. **Load Driver Class (Traditional Flow):** Traditional JDBC programs explicitly loaded the driver class using `Class.forName()`. With JDBC 4.0 and later, JDBC drivers can normally be discovered and loaded automatically through the Service Provider mechanism, so explicit `Class.forName()` is usually unnecessary when the driver is correctly available on the classpath.
 2. **Establish Connection:** Obtain a `Connection` object via `DriverManager.getConnection(url, user, password)`.
 3. **Create Statement:** Instantiate `Statement` or `PreparedStatement` from the `Connection`.
 4. **Execute SQL Query:** Call `executeQuery()` for `SELECT` or `executeUpdate()` for DML statements.
 5. **Close Connection:** Close JDBC resources (`ResultSet`, `Statement`, `Connection`) to release database locks and connection pools.
-

CHAPTER 2 — JDBC CORE API & SQL OPERATIONS

2.1 Driver Loading & `Class.forName()` (Traditional Flow vs Modern Discovery)

Traditional JDBC programs explicitly loaded the driver class using `Class.forName()`. With JDBC 4.0 and later, JDBC drivers can normally be discovered and loaded automatically through the Service Provider mechanism, so explicit `Class.forName()` is usually unnecessary when the driver is correctly available on the classpath.

2.2 `DriverManager` & `getConnection()`

`java.sql.DriverManager` acts as the management service connecting Java applications to database drivers.

```
String url = "jdbc:mysql://localhost:3306/school_db";
String user = "root";
String pass = "password";
Connection conn = DriverManager.getConnection(url, user, pass);
```

- **JDBC URL Syntax:** `jdbc:<subprotocol>://<host>:<port>/<database_name>`
- `jdbc`: Protocol
- `mysql`: Subprotocol (database vendor)
- `localhost:3306`: Host server and port
- `school_db`: Target database name

2.3 Statement vs PreparedStatement vs CallableStatement

Feature	Statement	PreparedStatement	CallableStatement
SQL Parameterization	Hardcoded string concatenation	Parameterized SQL (?) placeholders)	Stored procedure parameters
SQL Injection Protection	Vulnerable	Secure (separates query logic from data)	Secure
Execution Preparation	Compiled on every execution	Parameterized template (plan reuse depends on driver/DB)	Stored on DB server
Parameters	Direct string literals	Positional parameters (?)	IN, OUT, INOUT parameters

2.4 SQL Execution Methods & DML Operations (Syllabus 25.50 UPDATE & 25.51 DELETE)

- `executeQuery(String sql)`: Used exclusively for `SELECT` queries. Returns a `ResultSet` containing tabular records.
- `executeUpdate(String sql)`: Used for DML (`INSERT`, `UPDATE`, `DELETE`) or DDL (`CREATE`, `ALTER`, `DROP`) statements. Returns an `int` representing the count of affected rows (or `0` for DDL statements).
- `execute(String sql)`: Executes any SQL statement. Returns `boolean` (`true` if result is a `ResultSet`, `false` if update count or no result).

2.5 `ResultSet`, Cursor Navigation & Complete CRUD Concept (Syllabus 25.52)

A `ResultSet` maintains a cursor pointing to its current row of data:

- **Cursor State:** Initially positioned *before the first row*.
- `ResultSet.next()`: Moves the cursor down one row. Returns `boolean` (`true` if a valid row exists, `false` at EOF).
- **Data Accessor Methods:**
 - `getString(int colIndex) / getString(String colLabel)`
 - `getInt(int colIndex) / getInt(String colLabel)`
 - `getDouble(int colIndex) / getDouble(String colLabel)`
- *Note:* Column indices in JDBC are **1-based** (first column is `1`, not `0`).

2.6 SQL Injection Problem & `PreparedStatement` Protection

SQL Injection is a severe security vulnerability occurring when untrusted user input is directly concatenated into SQL query strings, altering the query's logical structure.

Dangerous Vulnerable Code (Using `Statement`):

```
// Vulnerable to SQL Injection
String query = "SELECT * FROM users WHERE username = '" + userInput + "' AND password = '" + passInput + "'";
// If userInput is: admin' --
// Resulting SQL: SELECT * FROM users WHERE username = 'admin' -- ' AND password = '...'
// Authenticates attacker without valid password!
```

Secure Code (Using `PreparedStatement`):

```
String sql = "SELECT * FROM users WHERE username = ? AND password = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, userInput); // Safe parameter binding
pstmt.setString(2, passInput);
ResultSet rs = pstmt.executeQuery();
```

- **Why `PreparedStatement` Prevents SQL Injection:** `PreparedStatement` is used to execute parameterized SQL statements using `?` placeholders. Parameter values are supplied separately from the SQL text, which helps prevent SQL injection when used correctly. Preparation and execution-plan reuse depend on the JDBC driver and database.

2.7 JDBC Transaction Management — Basic Concept, `commit()` & `rollback()` (Syllabus 25.54 & 25.55)

A **Transaction** is a logical unit of work consisting of one or more SQL operations that must either all succeed (ACID compliance) or all fail together.

- **Auto-Commit Mode:** By default, JDBC connections operate in auto-commit mode (`conn.getAutoCommit() == true`), where every individual SQL statement is committed automatically as an isolated transaction.
- **Disabling Auto-Commit:**

```
conn.setAutoCommit(false); // Begin manual transaction
```

- **Committing & Rolling Back:**

```
try {
    // Execute multiple update statements
    pstmt1.executeUpdate();
    pstmt2.executeUpdate();
    conn.commit(); // Permanently save changes
} catch (SQLException e) {
    conn.rollback(); // Undo all changes if error occurs
}
```

2.8 JDBC try-with-resources Resource Management (Syllabus 25.53)

Failing to close database connections, statements, and result sets causes database connection pool exhaustion and memory leaks.

- `java.sql.Connection`, `java.sql.Statement`, and `java.sql.ResultSet` all implement `java.lang.AutoCloseable`.
- Wrapping JDBC objects inside `try-with-resources` ensures they are closed automatically in reverse order of opening:

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement pstmt = conn.prepareStatement(sql);
    ResultSet rs = pstmt.executeQuery()) {
    while (rs.next()) {
        // Process records safely
    }
} // Resources automatically closed here
```

CHAPTER 3 — ORM, HIBERNATE & MAVEN

3.1 Object-Relational Impedance Mismatch & Persistence

- **Java Domain Model:** Object-oriented paradigm based on objects, classes, references, inheritance, encapsulation, and polymorphism.
- **Relational Database Model:** Relational paradigm based on tables, foreign keys, constraints, normalization, and SQL.
- **Impedance Mismatch:** The architectural mismatch between object-oriented data models and relational database tables.
- **Persistence:** The process of saving in-memory Java object states into a durable storage medium (RDBMS) so data survives application restarts.

3.2 ORM (Object-Relational Mapping) Concept

ORM (Object-Relational Mapping) is a programming technique that maps Java classes to database tables, instance fields to table columns, and object instances to table rows.

- Eliminates boiler-plate JDBC code (`ResultSet` extraction, manual parameter binding).
- Automatically generates and executes SQL statements based on object mapping metadata (annotations like `@Entity`, `@Table`, `@Id`).

3.3 Hibernate Introduction & Architecture

Hibernate is an open-source, high-performance ORM framework implementing the **Jakarta Persistence API (JPA)** specification.

- **Hibernate Core Components:**

- **SessionFactory:** A heavy-weight, thread-safe factory creating **Session** instances. Reads mapping configuration once during application startup.
- **Session:** A light-weight, non-thread-safe object representing a single unit of work/connection with the database. Used for CRUD operations (**save()**, **get()**, **update()**, **delete()**).
- **Transaction:** Manages database transactions explicitly (**tx.begin()**, **tx.commit()**).

3.4 JDBC vs Hibernate Comparison

Feature	Low-Level JDBC	Hibernate ORM Framework
Abstraction Level	Low-level SQL execution API	High-level Object-Relational Mapping framework
SQL Writing	Developer must write explicit SQL queries	Automatically generates database-specific SQL
Boilerplate Code	High (manual mapping, statement creation)	Low (automatic entity-to-table mapping)
Database Portability	Low (SQL syntax varies per vendor)	High (uses Dialects for seamless DB switching)
Caching	No built-in caching	Built-in First-Level (Session) and Second-Level caching

3.5 Hibernate Mini Application Configuration

A basic Hibernate entity mapped using JPA annotations:

```
import jakarta.persistence.*;
@Entity
@Table(name = "students")
public class StudentEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    @Column(name = "student_name", nullable = false)
    private String name;
    @Column(name = "email")
    private String email;
    // Default constructor required by Hibernate/JPA
    public StudentEntity() {}
    public StudentEntity(String name, String email) {
        this.name = name;
        this.email = email;
    }
    // Getters and Setters
    public int getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

3.6 Maven Introduction & Build Lifecycle

Apache Maven is a software project management and comprehension tool based on the **POM (Project Object Model)** concept.

- **Build Automation:** Automates compilation, testing, packaging (JAR/WAR generation), and deployment.
- **Dependency Management:** Automatically downloads required JAR libraries and their transitive dependencies from central repositories into a local repository (`~/.m2/repository`).

3.7 `pom.xml` Dependencies Structure

`pom.xml` is the central XML configuration file containing project configuration and external dependencies:


```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.fresher.app</groupId>
  <artifactId>jdbc-hibernate-demo</artifactId>
  <version>1.0.0</version>
  <dependencies>
    <!-- MySQL JDBC Driver -->
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.0.33</version>
    </dependency>
    <!-- Hibernate Core -->
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>6.2.7.Final</version>
    </dependency>
  </dependencies>
</project>

```

3.8 Standard Maven Directory Structure

Maven enforces a convention-over-configuration directory layout:

```

my-app/
  pom.xml                (Project Object Model file)
  src/
    main/
      java/              (Java source packages & code)
      resources/         (Configuration files: application.properties, hibernate.cfg.xml)
    )
    test/
      java/              (Unit test Java classes - JUnit)
      resources/         (Test environment configurations)

```

CHAPTER 4 — COMPLETE JDBC HANDS-ON PRACTICE

Common JDBC Coding Problems (Syllabus 25.56.1 to 25.56.18)

Problem 1: Establish Database Connection (25.56.1)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class EstablishConnectionDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/";
        String user = "root";
        String pass = "password";
        // Connecting to MySQL server
        try (Connection conn = DriverManager.getConnection(url, user, pass)) {
            if (conn != null) {
                System.out.println("Database connection established successfully!");
            }
        } catch (SQLException e) {
            System.err.println("Connection Failed: " + e.getMessage());
        }
    }
}
```

• **Expected Output:**

```
Database connection established successfully!
```

Problem 2: Create Database / Table (25.56.2)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
public class CreateDatabaseTableDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        String createTableSql = "CREATE TABLE IF NOT EXISTS students ( "
            + "id INT PRIMARY KEY AUTO_INCREMENT, "
            + "name VARCHAR(50) NOT NULL, "
            + "email VARCHAR(50) UNIQUE, "
            + "gpa DOUBLE)";
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            Statement stmt = conn.createStatement()) {
            stmt.executeUpdate(createTableSql);
            System.out.println("Table 'students' created successfully!");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

• **Expected Output:**

```
Table 'students' created successfully!
```

Problem 3: Insert Record (25.56.3) & Insert Multiple Records (25.56.4)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
public class InsertRecordsDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            Statement stmt = conn.createStatement()) {
            String sql1 = "INSERT INTO students(name, email, gpa) VALUES('Alice', 'alice@example.com', 3.8)";
            String sql2 = "INSERT INTO students(name, email, gpa) VALUES('Bob', 'bob@example.com', 3.5)";
            int rows1 = stmt.executeUpdate(sql1);
            int rows2 = stmt.executeUpdate(sql2);
            System.out.println("Inserted " + (rows1 + rows2) + " student records.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

- **Expected Output:**

```
Inserted 2 student records.
```

Problem 5: Select Records (25.56.5) & Display ResultSet (25.56.6)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class SelectDisplayResultSetDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        String query = "SELECT id, name, email, gpa FROM students";
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(query)) {
            System.out.println("ID | Name | Email | GPA");
            System.out.println("-----");
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String email = rs.getString("email");
                double gpa = rs.getDouble("gpa");
                System.out.println(id + " | " + name + " | " + email + " | " + gpa);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

• **Expected Output:**

```
ID | Name | Email | GPA
-----
1 | Alice | alice@example.com | 3.8
2 | Bob | bob@example.com | 3.5
```

Problem 7: Dynamic User Input (25.56.7), PreparedStatement Insert (25.56.8) & Select (25.56.9)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Scanner;
public class PreparedStatementDynamicDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        String insertSql = "INSERT INTO students(name, email, gpa) VALUES(?, ?, ?)";
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement pstmt = conn.prepareStatement(insertSql);
            Scanner scanner = new Scanner(System.in)) {
            System.out.print("Enter Student Name: ");
            String name = scanner.nextLine();
            System.out.print("Enter Student Email: ");
            String email = scanner.nextLine();
            System.out.print("Enter Student GPA: ");
            double gpa = scanner.nextDouble();
            // Setting positional parameter values
            pstmt.setString(1, name);
            pstmt.setString(2, email);
            pstmt.setDouble(3, gpa);
            int rows = pstmt.executeUpdate();
            System.out.println("Successfully inserted " + rows + " record dynamically.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

• **Expected Output (Sample):**

```
Enter Student Name: Charlie
Enter Student Email: charlie@example.com
Enter Student GPA: 3.9
Successfully inserted 1 record dynamically.
```

Problem 10: Update Record (25.56.10) & Delete Record (25.56.11)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
public class UpdateDeleteDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        String updateSql = "UPDATE students SET gpa = ? WHERE id = ?";
        String deleteSql = "DELETE FROM students WHERE id = ?";
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            PreparedStatement updatePstmt = conn.prepareStatement(updateSql);
            PreparedStatement deletePstmt = conn.prepareStatement(deleteSql)) {
            // 1. Update GPA for student ID 1
            updatePstmt.setDouble(1, 3.95);
            updatePstmt.setInt(2, 1);
            int updatedRows = updatePstmt.executeUpdate();
            System.out.println("Updated " + updatedRows + " record(s).");
            // 2. Delete student ID 2
            deletePstmt.setInt(1, 2);
            int deletedRows = deletePstmt.executeUpdate();
            System.out.println("Deleted " + deletedRows + " record(s).");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

• **Expected Output:**

```
Updated 1 record(s).
Deleted 1 record(s).
```

Problem 12: Complete CRUD Application (25.56.12), Student App (25.56.17) & Employee App (25.56.18)

```

import java.sql.*;
public class StudentCrudApp {
    private static final String URL = "jdbc:mysql://localhost:3306/school_db";
    private static final String USER = "root";
    private static final String PASS = "password";
    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            System.out.println("=== STUDENT CRUD SYSTEM ===");
            createStudent(conn, "David", "david@example.com", 3.7);
            readStudents(conn);
            updateStudentGpa(conn, 1, 4.0);
            deleteStudent(conn, 3);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
    private static void createStudent(Connection conn, String name, String email, double gpa) throws
    SQLException {
        String sql = "INSERT INTO students (name, email, gpa) VALUES (?, ?, ?)";
        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, name);
            pstmt.setString(2, email);
            pstmt.setDouble(3, gpa);
            pstmt.executeUpdate();
            System.out.println("[CREATE] Added: " + name);
        }
    }
    private static void readStudents(Connection conn) throws SQLException {
        String sql = "SELECT * FROM students";
        try (Statement stmt = conn.createStatement(); ResultSet rs = stmt.executeQuery(sql)) {
            System.out.println("[READ] All Students:");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id") + " | Name: " + rs.getString("name") + "
| GPA: " + rs.getDouble("gpa"));
            }
        }
    }
    private static void updateStudentGpa(Connection conn, int id, double newGpa) throws SQLException
    {
        String sql = "UPDATE students SET gpa = ? WHERE id = ?";
        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setDouble(1, newGpa);
            pstmt.setInt(2, id);
            pstmt.executeUpdate();
            System.out.println("[UPDATE] ID " + id + " GPA updated to " + newGpa);
        }
    }
    private static void deleteStudent(Connection conn, int id) throws SQLException {
        String sql = "DELETE FROM students WHERE id = ?";
        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, id);
            pstmt.executeUpdate();
            System.out.println("[DELETE] ID " + id + " deleted.");
        }
    }
}

```

• **Expected Output:**

```
=== STUDENT CRUD SYSTEM ===  
[CREATE] Added: David  
[READ] All Students:  
ID: 1 | Name: Alice | GPA: 3.95  
ID: 4 | Name: David | GPA: 3.7  
[UPDATE] ID 1 GPA updated to 4.0  
[DELETE] ID 3 deleted.
```

Problem 13: Batch Insert (25.56.13)

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.SQLException;  
public class BatchInsertDemo {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/school_db";  
        String user = "root";  
        String pass = "password";  
        String sql = "INSERT INTO students (name, email, gpa) VALUES (?, ?, ?)";  
        try (Connection conn = DriverManager.getConnection(url, user, pass);  
            PreparedStatement pstmt = conn.prepareStatement(sql)) {  
            // Adding Batch 1  
            pstmt.setString(1, "User-1");  
            pstmt.setString(2, "user1@example.com");  
            pstmt.setDouble(3, 3.2);  
            pstmt.addBatch();  
            // Adding Batch 2  
            pstmt.setString(1, "User-2");  
            pstmt.setString(2, "user2@example.com");  
            pstmt.setDouble(3, 3.6);  
            pstmt.addBatch();  
            // Executing batch SQL operations  
            int[] updateCounts = pstmt.executeBatch();  
            System.out.println("Batch execution complete. Statements processed: " + updateCounts.length);  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

• Expected Output:

```
Batch execution complete. Statements processed: 2
```

Problem 14: Transaction with commit() (25.56.14) & rollback() (25.56.15)


```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
public class TransactionCommitRollbackDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        String debitSql = "UPDATE accounts SET balance = balance - 500 WHERE account_id = 101";
        String creditSql = "UPDATE accounts SET balance = balance + 500 WHERE account_id = 102";
        try (Connection conn = DriverManager.getConnection(url, user, pass)) {
            // Disabling auto-commit to begin transaction block
            conn.setAutoCommit(false);
            try (PreparedStatement stmt1 = conn.prepareStatement(debitSql);
                PreparedStatement stmt2 = conn.prepareStatement(creditSql)) {
                stmt1.executeUpdate(); // Debit
                stmt2.executeUpdate(); // Credit
                // Committing transaction
                conn.commit();
                System.out.println("Transaction committed successfully!");
            } catch (SQLException e) {
                // Rolling back transaction on error
                conn.rollback();
                System.err.println("Error occurred! Transaction rolled back cleanly.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

• **Expected Output:**

```
Transaction committed successfully!
```

Problem 16: JDBC try-with-resources (25.56.16)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class TryWithResourcesJdbcDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school_db";
        String user = "root";
        String pass = "password";
        // Connection, Statement, and ResultSet automatically closed in reverse order
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT COUNT(*) AS total FROM students")) {
            if (rs.next()) {
                System.out.println("Total Students in DB: " + rs.getInt("total"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

• **Expected Output:**

```
Total Students in DB: 4
```

BOOK 7 END-OF-BOOK REVISION & INTERVIEW PREPARATION PACK

1. COMPLETE BOOK 7 REVISION SHEET

- **JDBC Driver Types:** Type 4 Pure Java Native-Protocol Driver communicates directly with the database using its native protocol without native client libraries.
- **Key API Objects:** `DriverManager` $\rightarrow$ `Connection` $\rightarrow$ `Statement` / `PreparedStatement` $\rightarrow$ `ResultSet`.
- **PreparedStatement:** Used to execute parameterized SQL statements using `?` placeholders. Parameter values are supplied separately from the SQL text, which helps prevent SQL injection when used correctly. Preparation and execution-plan reuse depend on the JDBC driver and database.
- **ResultSet Navigation:** Cursor starts *before first row*. Call `rs.next()` to advance. Column indexing is **1-based**.
- **Transactions:** Disable auto-commit via `conn.setAutoCommit(false)`, perform updates, call `conn.commit()` on success, or `conn.rollback()` inside `catch` block.
- **ORM & Hibernate:** Maps Java entities to database tables using JPA annotations (`@Entity`, `@Table`, `@Id`). `SessionFactory` creates `Session` objects for persistence operations.
- **Maven:** Build automation tool utilizing `pom.xml` for dependency management.

2. IMPORTANT FRESHER INTERVIEW QUESTIONS

1. Why is `PreparedStatement` preferred over `Statement` in JDBC?

- **Answer:** `PreparedStatement` is used to execute parameterized SQL statements using `?` placeholders. Parameter values are supplied separately from the SQL text, which helps prevent SQL injection when used correctly. Preparation and execution-plan reuse depend on the JDBC driver and database.

2. How do you execute transactions in JDBC?

- **Answer:** Disable auto-commit mode using `conn.setAutoCommit(false)`. Execute all related SQL statements inside a `try` block and call `conn.commit()`. In the `catch(SQLException)` block, execute `conn.rollback()` to undo uncommitted changes if any statement fails.

3. What is the difference between `execute()`, `executeQuery()`, and `executeUpdate()`?

- **Answer:**
- `executeQuery()` executes `SELECT` statements and returns a `ResultSet`.
- `executeUpdate()` executes DML (`INSERT`, `UPDATE`, `DELETE`) or DDL statements and returns an `int` row count.
- `execute()` executes any SQL statement and returns `boolean` (`true` for `ResultSet`, `false` for update count).

4. What is the purpose of Maven's `pom.xml` file?

- **Answer:** `pom.xml` (Project Object Model) contains project details, configuration settings, plugins, and dependencies. Maven reads `pom.xml` to automatically download required library JARs from central repositories into local storage (`~/.m2/repository`).

3. IMPORTANT OUTPUT-PREDICTION QUESTIONS

Question 1: `ResultSet` 1-Based Indexing

```
// Assuming ResultSet 'rs' contains 1 row with column 'id' = 10 and 'name' = "Java"
if (rs.next()) {
    System.out.println(rs.getString(1) + " - " + rs.getString(2));
}
```

- **Output:** `10 - Java`
- **Explanation:** JDBC column indexing starts at `1`, not `0`. `rs.getString(1)` retrieves the first column (`id`), and `rs.getString(2)` retrieves the second column (`name`).

4. DEBUGGING PRACTICE & COMMON MISTAKES

1. SQL Injection Vulnerability via String Concatenation

- **Bug:** Concatenating raw user inputs into `Statement` strings.
- **Fix:** Replace `Statement` with `PreparedStatement` using positional `?` placeholders.

2. Connection Leaks and Memory Exhaustion

- **Bug:** Opening connections without closing them in `finally` or `try-with-resources`.
- **Fix:** Wrap `Connection`, `Statement`, and `ResultSet` in `try-with-resources`.

5. FRESHER-LEVEL MOCK INTERVIEW SCENARIO

Interviewer: Can you explain how JDBC prevents SQL Injection?

Candidate: Yes, sir! SQL Injection occurs when untrusted user inputs are concatenated directly into SQL query strings, allowing attackers to manipulate query logic. JDBC prevents this by using `PreparedStatement`. `PreparedStatement` is used to execute parameterized SQL statements using `?` placeholders. Parameter values are supplied separately from the SQL text, which helps prevent SQL injection when used correctly. Preparation and execution-plan reuse depend on the JDBC driver and database.

Interviewer: Excellent! What is the role of Hibernate in a Java Full Stack application?

Candidate: Hibernate is an ORM framework that eliminates low-level boilerplate JDBC code. It maps Java entity classes directly to database tables using JPA annotations, generating vendor-specific SQL automatically and providing built-in caching and transaction management.

6. CONCEPT COMPARISON TABLES

Feature	Low-Level JDBC	Hibernate ORM
SQL Writing	Manual SQL statements	Auto-generated SQL via HQL/Criteria/JPA
Mapping	Manual <code>ResultSet</code> mapping	Automatic Entity class mapping (<code>@Entity</code>)
Portability	Low (vendor SQL differences)	High (handled via Database Dialects)

7. COMPLETE BOOK 7 SYLLABUS COVERAGE CHECKLIST

- [x] **Section 25: JDBC & Database Connectivity (25.1 to 25.56.18)**
- [x] Section 25.48: JDBC Detailed Architecture
- [x] Section 25.49: JDBC Driver Types (1 to 4)
- [x] Section 25.50: UPDATE Operation
- [x] Section 25.51: DELETE Operation
- [x] Section 25.52: Complete CRUD Operation
- [x] Section 25.53: JDBC `try-with-resources`
- [x] Section 25.54: Transaction — Basic Concept
- [x] Section 25.55: `commit()` / `rollback()` — Basic
- [x] Web Architecture, Driver Loading (Traditional vs JDBC 4+ SPI Discovery)
- [x] `Connection`, `Statement`, `PreparedStatement`, `CallableStatement`, `ResultSet`
- [x] SQL Injection protection, Transaction Management (`commit`, `rollback`)
- [x] ORM Concept, Entity, `@Entity`, `@Table`, `@Id`, `SessionFactory`, `Session`, `Transaction`

- [x] Maven Introduction, [pom.xml](#), Dependencies, Directory Structure
 - [x] All 18 JDBC coding problems (25.56.1 to 25.56.18) solved with complete code and notes.
-